

Why This Course, and Why Python

DSA27 — Lecture 01 · CS201 Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

22 September 2026

Contents

Before We Start	2
Where everything lives	2
The one thing to do today	2
Programming Languages Are Not the Same	3
The argument that sounds right	3
Why it is wrong	3
A language is a set of decisions	3
The decisions have owners	4
A correction I owe you	4
Paradigms: the same job, three ways	5
Type systems: two axes, not one	6
Memory: who cleans up?	6
Comparing Languages Honestly	7
What are you actually asking?	7
The rankings disagree — and that is the lesson	7
Ecosystem beats syntax	8
Why Python for This Course	8
The honest case for Python	8
The honest case against Python — for us	8
From a Variable to a Data Structure	9
The ladder	9
Abstract Data Type vs implementation	10
Know-How	10
Cost Is the Whole Point	11
What an algorithm is	11
Measure, do not assert	11
The gap is not small	12
The Course Itself	12
CS201, officially	12
How you pass	13
How we will work	13

Homework 1	14
Due before the next lecture	14
Summary	14
Six things to keep	14
Next	14
Sources and further reading	15

How to read this document. This is the handout for Lecture 01. It contains everything that was on the slides, plus the parts I said out loud and the sources for every claim. If a statement comes from the faculty bylaw, the page number is printed next to it so you can check it yourself. You should check it yourself.

Slides: DSA27-L01-slides.pdf · Repository: <https://github.com/helghareeb/DSA27>

Before We Start

Where everything lives

What	Where
Code, exercises, these notes	https://github.com/helghareeb/DSA27
Course updates, announcements	WhatsApp channel
Some — not all — video lectures	https://www.youtube.com/@0xHGH

Announcements go to the WhatsApp channel first. The repository is the source of truth for anything technical: if the channel and the repository disagree, the repository is right and I have made a mistake in the channel.

The QR code for the channel is in docs/assets/whatsapp-channel-qr.jpg, and on the links page, docs/links.md.

The one thing to do today

```
git clone https://github.com/helghareeb/DSA27.git
cd DSA27
py -3.13 -m venv .venv
.\.venv\Scripts\activate
pip install -r requirements.txt
pytest -m "not challenge"
```

Every one of those tests must pass **before** you write a line of course code.

Those tests do not check your algorithms. They check your machine: your interpreter version, that you are inside a virtual environment, the plotting stack, the Graphviz dot binary, and the drawing helpers in viz/. When one fails, the message names exactly what is missing. Fix that first. Debugging a linked list on a broken environment is a waste of your evening.

Full setup instructions, including the Windows “Microsoft Store opens instead of Python” trap, are in the repository `README.md`.

Programming Languages Are Not the Same

The argument that sounds right

Every language can print Hello, World.

Every language can add two numbers.

Every language is, in the end, reduced to instructions the same processor executes.

So all programming languages are the same. Only the syntax differs.

This is the most common thing an intelligent second-year student believes, and I want to spend real time on it, because the rest of the course depends on taking it apart.

The argument is not stupid. It has a true premise: in terms of *what can be computed*, the languages you will meet are equivalent — they are all Turing-complete, so any program you can write in one, you can write in another. That is a genuine theorem and it is worth knowing.

The error is in the word “same”. Turing-equivalence tells you the set of computable functions is identical. It tells you **nothing** about what the code costs to write, what it costs to read six months later, what it costs to run, which mistakes it lets you make silently, or how many people can maintain it.

Why it is wrong

Turing-equivalence says: **the same problems are solvable**.

It does not say they are solvable:

- at the same **cost**
- by the same number of **people**
- in the same amount of **time**
- with the same number of **mistakes surviving to production**

A language is not a neutral pipe you pour logic through. A language is a set of **decisions** — about what is easy, what is hard, what is forbidden, and what fails loudly versus what fails silently. Those decisions were made by people, for reasons, and the reasons are usually written down.

A language is a set of decisions

Programming Language → Design → Philosophy

Ask of any language: **what did its designers refuse to do, and why?**

That chain — language, design, philosophy — is the one I drew on the board. Read it right to left and it explains the language. Read it left to right and it explains the code you are looking at.

The decisions have owners

Person	Language	The decision, in one line
Guido van Rossum	Python	Readability is worth paying for.
Bjarne Stroustrup	C++	Leave no room for a lower-level language.
Graydon Hoare	Rust	Memory safety without a garbage collector.
Pike, Thompson, Griesemer	Go	Fewer features, faster builds, at scale.

Python. Van Rossum’s priority was that code is read far more often than it is written. The consequences are everywhere: significant indentation (you cannot write badly-indented Python, because badly-indented Python is a *different* program), one obvious way to do things, a small keyword set. Run `import this` in a Python prompt and read the nineteen lines that come out — that is *PEP 20, The Zen of Python*, the language’s philosophy stated by its own community.

C++. Stroustrup’s rule was that C++ should not leave a gap underneath it that forces you down to assembly or C. Hence: you pay for what you use and nothing else, you get direct control of memory, and the language will let you do almost anything — including destroy yourself. The philosophy produces both the performance and the danger.

Rust. Hoare’s question was whether memory safety requires a garbage collector. Rust’s answer — ownership and borrowing, checked at compile time — is a genuinely new point in the design space, and it is why Rust has been the “most admired” language in the Stack Overflow survey for years running.

Go. Deliberately *small*. Its designers left out features other languages consider essential, on the grounds that a large team compiling a large codebase benefits more from simplicity and build speed than from expressiveness.

Four different answers. None of them is wrong. They are answers to **different questions**.

A correction I owe you

On the board I put **Donald Knuth** next to Stroustrup and van Rossum. That was loose, and I want to fix it.

- Knuth did not design a language you will write production code in.
- He wrote *The Art of Computer Programming*, created **TeX**, and invented **literate programming**.
- His subject is **algorithms** — their analysis, their correctness, their cost.

He is not the bridge to languages. He is the bridge to this course.

I am correcting this deliberately and in public, because it is a habit I want you to copy. When you find out that something you said was wrong, you say so plainly, you fix it, and you move on.

You do not quietly delete it and hope nobody noticed. In engineering, a person who corrects themselves is more trustworthy than a person who is never wrong, because the second person does not exist.

And the correction is useful here. Knuth's life's work is the question this course asks: *given a problem, what is the best way to organise the data, what does the resulting algorithm cost, and how do you prove it?* The famous line is his:

"Premature optimization is the root of all evil (or at least most of it) in programming." — Donald Knuth, Structured Programming with go to Statements, 1974

Note the word **premature**. It is usually quoted by people who want permission never to think about performance. Knuth's actual point, in context, is that you should not micro-optimize the 97% of code that does not matter — *so that* you can spend real effort on the 3% that does. Finding that 3% is what complexity analysis is for. That is Week 2.

Paradigms: the same job, three ways

Sum the squares of the even numbers in a list.

```
# Procedural – describe the steps
total = 0
for n in numbers:
    if n % 2 == 0:
        total += n * n

# Functional – describe the transformation
total = sum(map(lambda n: n * n, filter(lambda n: n % 2 == 0, numbers)))

# Declarative (comprehension) – describe the result
total = sum(n * n for n in numbers if n % 2 == 0)
```

Same output. Same language. **Three different ways of thinking.**

A **paradigm** is not a feature list, it is a way of decomposing a problem. The four you will meet:

- **Procedural / imperative** — the program is a sequence of steps that change state. C, Pascal, and the inside of almost every Python function.
- **Object-oriented** — the program is objects that own their data and expose operations on it. Java, C#, C++, and Python when you write a `class`. *This is how dsa/ is built: a `Stack` owns its storage and exposes `push`, `pop` and `peek`. You should not be able to reach inside it, and neither should its users.*
- **Functional** — the program is the composition of functions, and you avoid changing state. Haskell, Elixir, Lisp; and `map`, `filter` and closures in Python.
- **Declarative / logic** — you state *what* you want and the system works out *how*. SQL, regular expressions, CSS, Prolog.

Most languages you will use are **multi-paradigm**, and Python aggressively so. That is a strength and a trap: Python will let you write all three styles above, so *you* have to decide which one makes

the code clearest. The third version is the one I would accept in a review. It is shortest, it says what it means, and it builds no intermediate list.

Type systems: two axes, not one

Students collapse these into one question. They are independent.

	Static — checked at compile time	Dynamic — checked at run time
Strong — refuses to guess	Java, C#, Rust, Haskell	Python , Ruby
Weak — converts silently	C, C++	JavaScript, PHP

Static vs dynamic is about **when** types are checked. **Strong vs weak** is about **how strictly** types are kept apart.

Python is **dynamically** typed — a name can be bound to an `int` now and a `str` later — but **strongly** typed, which is why this raises an error instead of guessing:

```
>>> "3" + 5
TypeError: can only concatenate str (not "int") to str
```

JavaScript, which is dynamic and weak, answers `"35"`. Neither behaviour is a bug. They are different decisions about whether a surprising conversion is more useful than a loud failure. Python's answer — fail loudly — is the one that will save you in this course, because a silent type coercion inside a sorting routine is genuinely hard to find.

One more thing worth knowing: Python has **optional type hints** (`def push(self, value: int) -> None:`). They are not enforced at run time — the interpreter ignores them — but tools read them, and so do humans. The skeletons in `dsa/` use them. Treat them as documentation your editor can check.

Memory: who cleans up?

- **Manual** — you allocate, you free. C, C++. Fastest, and the source of decades of security vulnerabilities.
- **Garbage collected** — the runtime frees what you can no longer reach. Python, Java, C#, Go. Safe, at the cost of control and some pauses.
- **Ownership** — the compiler proves at build time that memory is freed exactly once. Rust. Safe *and* fast, at the cost of a harder compiler to satisfy.

This matters to us more than it looks. When you implement `dsa/linked_list.py` and you “remove” a node, in C you would call `free()`, and a mistake there is a crash or a security hole. In Python you simply stop referring to the node and the garbage collector deals with it.

That convenience is exactly what makes the structure hard to see. You will build a linked list without ever touching a pointer. So use the debugger — set a breakpoint in `dsa/linked_list.py`, press **F5**, choose *pytest: current test file*, and watch `node.next` change in the Variables panel as you step with **F10**. The repository README has the setup.

Seeing a reference move beats any diagram, including the ones I drew.

Comparing Languages Honestly

What are you actually asking?

“Which language is best?” is not a question. These are questions:

- Best **for what task**? (a web service, embedded firmware, data analysis, a game)
- Best **for whom**? (you, alone, learning — or forty engineers for ten years)
- Best by **which measure**? (speed to write, speed to run, hiring pool, safety)
- Best **when**? (this project, or the one you maintain until 2035)

Change any one of those and the answer changes. A language that is perfect for a weekend data-analysis script is a poor choice for firmware on a device with 64 KB of RAM, and the reverse is equally true. This is not diplomacy, it is engineering.

The image I put on the board is the point: a hammer, surrounded by bent nails. A hammer is an excellent tool. The nails are bent because the hammer was the only tool the person owned. Learn more than one language — and the reason is not the second language itself. It is that you cannot see the decisions your first language made until you have seen a different set.

The rankings disagree — and that is the lesson

Index	What it actually counts	Crowns
TIOBE	Search-engine hits for “X programming”	Python
Stack Overflow Survey	Self-reported usage by respondents	JavaScript
GitHub / Octoverse	Commits and repositories	TypeScript / Python
PYPL	Google searches for “X tutorial”	Python

Different instruments. Different answers. **None of them measures quality.**

TIOBE, September 2026 — Python 17.76%, C 10.28%, C++ 8.67%, Java 7.54%, C# 4.22%, JavaScript 2.76%, Visual Basic 2.55%. Rust sits at number 10.

Read those numbers carefully. TIOBE counts *search-engine results*, so it is a proxy for how much people write and ask about a language — not how much code is running in production, and certainly not how good the language is. Visual Basic ranking above Rust should tell you everything about what the index is and is not.

Compare: the Stack Overflow Developer Survey has put **JavaScript** first in self-reported usage nearly every year since 2011, while **Rust** has been the *most admired* language — roughly 72% of the developers who use it want to keep using it. Usage and admiration are different questions, measured differently, with different answers.

So what do you do with the rankings? Two things, and only two:

1. Use them to see **trends over years**, not positions in a month. The interesting fact is not that Python is first this September — it is that it climbed there and stayed.
2. Use them to check that a language is **not dying**, because a language with no community has no libraries, no answers and no jobs.

Never use them to pick a language for a task. For that, you read about the task.

All figures above are dated **September 2026** and will be wrong soon. Any slide that quotes a ranking without a date is lying to you by omission — including, I will admit, some of my own older slides.

Ecosystem beats syntax

What you actually adopt when you adopt a language:

- **Libraries** — has someone already solved your problem, well?
- **Tooling** — debugger, profiler, package manager, formatter, test runner
- **Community** — is your question already answered?
- **Interoperability** — can it call the fast thing written in something else?
- **Longevity** — will it still be maintained in ten years?

Syntax is the part of a language you stop noticing after two weeks. The ecosystem is the part you live inside for years. When a team argues about which language to use, this list is what they are really arguing about, even when they think they are arguing about semicolons.

Why Python for This Course

The honest case for Python

- **Where the field is.** Research, scientific computing, data analysis, and essentially all AI/ML/DL tooling.
- **Bindings.** The heavy numerical work is C, C++, CUDA or Fortran underneath; Python is the layer you steer it from. NumPy is not slow Python — it is fast C with a Python handle on it.
- **Reading cost.** Python is close enough to pseudocode that an algorithm survives the translation onto the page. That matters in a course where the algorithm *is* the content.
- **You will meet it again.** In every one of your programs — AI, Medical Informatics, Software Engineering.

That “bindings” point is the arrow I drew from the Python cloud across to the production box, and it is the reason Python occupies the position it does. A language that is easy to write but slow to run would be a toy. A language that is easy to write and can *call* the fast thing is infrastructure.

The honest case against Python — for us

Python already contains, as built-ins, most of what this course studies.

You will build	Python already has
Dynamic array	<code>list</code>
Hash map	<code>dict</code>
Stack / queue	<code>list</code> , <code>collections.deque</code>
Sorting	<code>sorted()</code> , <code>list.sort()</code> — Timsort
Priority queue	<code>heapq</code>

The tool hides exactly the thing we are here to look at.

This is a real drawback and I will not pretend otherwise. It is also precisely why the rule in `dsa/__init__.py` exists:

No module may use the built-in it is reimplementing.

So `dsa/dynamic_array.py` may not be backed by a `list` — it uses a raw `ctypes` block, and `make_block()` is given to you because allocating raw memory is not the lesson. `dsa/hashmap.py` may not be backed by a `dict`. If you route around this rule the tests may even pass, and you will have learned nothing. The tests are not the point. They are the *evidence* for the point.

Python is the vehicle of this course, not its subject. Everything you build here — the cost of an insertion, why a hash map degrades, why quicksort's pivot matters — transfers unchanged to C++, Java, Rust or Go. The syntax is the only part that does not transfer, and the syntax is the cheap part.

From a Variable to a Data Structure

The ladder

variable → object → class → collection → **data structure**

```
i = 42                # a name bound to an object
s = "hello"           # an object of class str
xs = [42, "hello", 3.14] # a collection of references
```

In Python a variable is **not a box holding a value**. It is a **name bound to an object**.

This is the ladder I drew, and it is worth climbing slowly.

In C, `int i = 42;` reserves four bytes and puts the number in them. The name *is* the storage.

In Python, `i = 42` creates (or finds) an integer object somewhere in memory and binds the name *i* to it. The name is a label, not a container. Two consequences you will trip over this term:

```
a = [1, 2, 3]
b = a                # b is not a copy. Both names point at ONE list.
b.append(4)
print(a)             # [1, 2, 3, 4]
```

and:

```
xs = [42, "hello", 3.14]
```

A Python list does not store the objects; it stores **references** to them, which is how one list holds three different types. When you build `dsa/dynamic_array.py` on a raw `ctypes` block you will be storing references too — and you will see the machinery that list normally hides.

Abstract Data Type vs implementation

Abstract Data Type (ADT) — النوع المجرد للبيانات The **contract**: what operations exist, what they mean, and what they cost. *Nothing* about how the data is stored.

Data structure — هيكل البيانات A concrete **arrangement of data in memory** that fulfils the contract.

One ADT, many structures. The ADT **Stack** says only: push adds, pop removes the most recent, peek looks, all in $O(1)$. That contract can be fulfilled by a dynamic array or by a linked list. Both are honest stacks. They differ in cache behaviour, in memory overhead per element, and in whether a single operation can occasionally be slow.

Choosing between them **is the engineering**, and you cannot choose without knowing the cost of each — which is why complexity comes before the structures, in Week 2.

A short glossary, since I will use both languages all term:

English	العربية
Data structure	هيكل البيانات
Algorithm	الخوارزمية
Abstract Data Type	النوع المجرد للبيانات
Complexity	التعقيد
Array	المصفوفة
Linked list	القائمة المتصلة
Stack	المكدس
Queue	الطابور
Tree	الشجرة
Graph	الرسم البياني
Hash table	جدول التجزئة
Sorting	الترتيب
Searching	البحث
Recursion	الاستدعاء الذاتي

Know-How

Three different things, and only the third is worth much:

1. **Know-what** — you can define a stack. *An hour's reading.*
2. **Know-how** — you can build one that works. *This course's exercises.*
3. **Know-when and know-why** — you can say why a stack and not a queue, and what it costs. *This course's exams, and your career.*

Know-How is the word I wrote on the board and circled, and it is the spine of this course.

The reason `dsa/` is a set of skeletons that all raise `NotImplementedError` is level 2. The reason the exam will ask you *which structure and why* rather than *define a heap* is level 3.

A student who has memorised that a hash map is “ $O(1)$ average” has level 1. A student who can implement one has level 2. A student who can explain why their hash map degraded to $O(n)$ on real data, and then fix it, has level 3 — and is employable.

Cost Is the Whole Point

What an algorithm is

An **algorithm** is a finite, unambiguous procedure that turns an input into the correct output.

We care about three properties, in this order:

1. **Correctness** — does it produce the right answer, always?
2. **Cost in time** — how does the work grow as the input grows?
3. **Cost in space** — how does the memory grow as the input grows?

Correctness first. A fast wrong answer is worthless, and “it worked on my three test cases” is not correctness. This is why the exercises in this course are delivered as **tests**: tests/ defines what “correct” means, in a form a machine can check, before you write a line of the implementation.

Measure, do not assert

Do not take my word that binary search beats linear search. **Measure it.**

```
from viz.complexity import measure, plot_growth
import random

sizes = [1000, 2000, 4000, 8000, 16000]
make = lambda n: (sorted(random.random() for _ in range(n)), 0.5)

plot_growth(
    {"linear": measure(lambda a: linear_search(*a), sizes, make),
     "binary": measure(lambda a: binary_search(*a), sizes, make)},
    reference=["n", "log n"],
)
```

`viz/complexity.py` is already written for you — it is infrastructure, not an exercise. `measure()` times a function over a range of input sizes and returns the best time at each size; `plot_growth()` draws your measurements against scaled reference curves for 1, $\log n$, n , $n \log n$, n^2 , n^3 and 2^n .

Notice what the picture gives you that a table of Big-O does not: the **crossover point**. Linear search often beats binary search on small inputs, because binary search does more work per step and the constant factors dominate. Big-O deliberately throws those constants away. That is what makes it useful at scale and misleading at $n = 20$.

This is the habit I want. When you are unsure about cost, do not argue and do not guess. Measure it, plot it, and look.

The gap is not small

n	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(2^n)$
10	3	10	33	100	1,024
100	7	100	664	10,000	$\sim 1.3 \times 10^{30}$
1,000	10	1,000	9,966	1,000,000	—
1,000,000	20	10^6	$\sim 2 \times 10^7$	10^{12}	—

At $n = 1,000,000$ the $O(\log n)$ algorithm does **20** steps. The $O(n^2)$ one does a **trillion**.

Read the last row until it is uncomfortable. This is not an academic distinction and it is not something a faster machine fixes. If your algorithm is $O(n^2)$ and your data grows tenfold, your runtime grows a hundredfold; buying a computer twice as fast buys back a factor of two.

Choosing the right structure is how you move between rows of that table. That is the entire course, stated in one table.

The Course Itself

CS201, officially

Code	CS201
Name	Data Structures and Algorithms — هياكل البيانات والخوارزميات
Credit hours	3 (Lecture 2 · Tutorial — · Laboratory 2)
Prerequisite	Introduction to Programming and Problem Solving (CS101)
Level	Sophomore — Fall (المستوى الثاني)
Student workload	155 hours · 5.14 ECTS
Taught as a major in	IS, CS, IT, SE, MI, AI , NCS

Source: *FCIS Internal Bylaw, Credit Hour System, 2026*, p. 138 (course specification) and p. 36 (study plan). The bylaw itself is in the repository at [docs/course/regulations/FCIS-Bylaw-2026.pdf](#).

Note the last row: CS201 is a **major requirement in all seven programs** of the faculty. Whatever you are enrolled in, this course is not optional and not a service course — it is core.

What it unlocks: **IS202 Introduction to Database Systems** takes CS201 as a prerequisite, and so does **CS303 Analysis and Design of Algorithms** in Junior Fall, which is where sorting lower bounds, graph algorithms, NP-completeness and dynamic programming are treated properly. This course is the foundation that one is built on.

How you pass

Component	Weight
Student Activities / Practical Exam	20
Midterm	15
Oral	5
Final written exam	60

Two hard rules from the bylaw:

- **≥ 60% overall and ≥ 30% of the final exam mark** — both, or you fail.
- **≥ 75% attendance** of lectures and labs, or you are **محروم** and cannot sit the final at all.

Sources: assessment weights, *FCIS Bylaw 2026*, p. 138 — the CS201 specification itself. Pass rule, Article 17, p. 23. Attendance and حرمان Article 15, p. 21. Final exam duration is 2 hours, pp. 23–24.

Read the 30% rule again, because students lose years to it: someone who collects 35 of the 40 coursework marks and then scores 25% on the final has **failed**, regardless of the total. Coursework cannot rescue a final exam below 30%.

The attendance rule is not a threat, it is arithmetic. 75% of a 14-week course means you can miss roughly three sessions. Not five.

Grade scale (*FCIS Bylaw 2026*, p. 22): A+ ≥97% (4.0) · A 93–97 (4.0) · A– 89–93 (3.7) · B+ 84–89 (3.3) · B 80–84 (3.0) · B– 76–80 (2.7) · C+ 73–76 (2.3) · C 70–73 (2.0) · C– 67–70 (1.7) · D+ 64–67 (1.3) · D 60–64 (1.0) · F <60% (0.0).

Note on the older bylaws. The 2013, 2014 and 2020 program bylaws in `docs/course/regulations/` disagree with each other on small points — the value of A+ (4.00 vs 4.33) and the wording of the final-exam weight. **The 2026 faculty bylaw governs, and it is the one quoted above.** The discrepancies are documented in `docs/course/regulations/dsa-in-your-program.md` rather than hidden.

How we will work

- **Lecture** — the idea, the cost, and why it exists.
- **Lab** — you implement it. The skeletons in `dsa/` all raise `NotImplementedError` on day one. That is the assignment.
- **Tests** — `tests/` defines “correct”. `pytest -m challenge` runs every exercise.
- **Visualisation** — `viz/` draws arrays, lists, trees and graphs, animates your sorts, and plots your measured complexity. It never imports `dsa/`, so it works with whatever you write.

They all fail on day one. That is the point, and it is the honest version of a course: you are not asked to reproduce my solution, you are asked to satisfy a specification.

The full 14-week plan, with the module and test file for each week, is in `docs/course/01-study-plan.md`.

Homework 1

Due before the next lecture

1. **Set up your environment.** Clone the repository, create the virtual environment, install the requirements, and make `pytest -m "not challenge"` pass. Bring the output.
2. **Read** Peter Norvig, *Teach Yourself Programming in Ten Years* — <https://norvig.com/21-days.html>. It is short.
3. **Run** `import this` in a Python prompt and read *The Zen of Python*. Pick the line you disagree with most, and be ready to say why.
4. **Write — one page, no more.** Choose two languages you have used or heard about. Compare them on **three criteria you choose yourself**, say which you would pick for a **specific named task**, and state what you are giving up by picking it.

On item 2: Norvig's essay is the antidote to "Learn X in 21 Days". His argument is that expertise takes roughly ten years of deliberate practice, and that the books promising otherwise are selling something. I put this on the last slide of a talk seven years ago and I am putting it in the first lecture of this course, because the sooner you stop looking for the shortcut, the sooner you start walking the road.

On item 4: there is no correct answer and I am not looking for one. I am looking for whether you can state a criterion, apply it, and be honest about the cost. A paragraph ending "...so I would choose Go, and I am giving up the library ecosystem I would have had in Python" earns full marks. A paragraph ending "...so Python is the best language" earns very few.

Summary

Six things to keep

1. Languages are **not** the same. Each is a set of decisions with an author and a reason.
2. "Which is best?" is unanswerable until you say **best for what, for whom, by which measure**.
3. Popularity indices measure **different things** and disagree. None measures quality. Always check the date.
4. We use Python because it reads like the algorithm — and we **reimplement its built-ins** because it otherwise hides the subject.
5. An **ADT** is a contract; a **data structure** is one way to keep it. Choosing between them is the engineering.
6. **Know-how** beats know-what; **know-why** beats both.

Next

Week 2 — Complexity. How to say what an algorithm costs, and how to *measure* whether you were right.

Before then: `make pytest -m "not challenge" pass`.

Sources and further reading

Official

- *FCIS Internal Bylaw for the Bachelor Stage, Credit Hour System, 2026* — docs/course/regulations/FCIS Bylaw-2026.pdf. CS201 specification p. 138; Sophomore Fall study plan p. 36; grade scale p. 22; pass rule p. 23; attendance p. 21.
- Program bylaws for Software Engineering (2013), Medical Informatics (2014) and Artificial Intelligence (2020) — same folder. Superseded; kept for reference.

Read this term

- Peter Norvig, *Teach Yourself Programming in Ten Years* — <https://norvig.com/21-days.html>
- Tim Peters, *PEP 20* — *The Zen of Python* — <https://peps.python.org/pep-0020/>
- Cormen, Leiserson, Rivest & Stein, *Introduction to Algorithms* (CLRS), 4th ed., MIT Press, 2022 — the reference.
- Sedgewick & Wayne, *Algorithms*, 4th ed. — gentler, excellent figures.
- Goodrich, Tamassia & Goldwasser, *Data Structures and Algorithms in Python* — closest to what we do here.

Data quoted in this lecture

- TIOBE Index, September 2026 — <https://www.tiobe.com/tiobe-index/>
- Stack Overflow Developer Survey — <https://survey.stackoverflow.co/>

All figures are as of **22 September 2026** and will need rechecking.

This document replaces "Programming Languages are Not the Same" (22 September 2019). The argument survives; the data, the links and one attribution did not.